WOOT!!!

```
#!/usr/bin/env python3
"""
SpiralCore Cantor-Abraxas Architecture - Interactive REPL App
Contains all 6 Primitives + Command Prompt Shell + Geodesic Router
No base64 blobs, no zip files, no extra dependencies!
"""

import sys
import os
import math
import time

try:
import numpy as np
except ImportError:
print("Numpy is required for vector operations in PyDroid 3.")

#
============================================================================
=
# PRIMITIVE 1: Mode A 3-Adic Carry Engine
#
============================================================================
=
class ThreeAdicCarryEngine:
def __init__(self, max_roots_bound: int = 16):
self.N = 0
```

```
self.max_roots_bound = max_roots_bound
self.active_roots = []

@staticmethod
def valuation_3(n: int) -> int:
if n <= 0:
return 0
v = 0
while n % 3 == 0:
v += 1
n //= 3
return v

def step(self, payload_value: float = 1.0) -> dict:
self.N += 1
v3 = self.valuation_3(self.N)
delta_chi = 1 - 2 * v3

self.active_roots.append({"id": self.N, "value": payload_value, "valuation": v3})

if v3 > 0 and len(self.active_roots) >= 3:
num_to_fold = min(len(self.active_roots), 3 ** v3)
if num_to_fold >= 2:
folded = self.active_roots[-num_to_fold:]
self.active_roots = self.active_roots[:-num_to_fold]
avg_val = sum(r["value"] for r in folded) / len(folded)
self.active_roots.append({"id": f"folded_{self.N}", "value": avg_val, "valuation":
v3})

beta_0 = len(self.active_roots)
beta_0_bound = min(self.max_roots_bound, 2 * int(math.floor(math.log(self.N +
1, 3))) + 2)
```

```
return {
"N": self.N,
"v_3": v3,
"delta_chi": delta_chi,
"beta_0": beta_0,
"beta_0_bound": beta_0_bound,
"active_roots_count": len(self.active_roots)
}

#
================================================================
=
# PRIMITIVE 2: 4D Kuramoto Phase Integrator
#
================================================================
=
class KuramotoPhaseIntegrator:
def __init__(self, num_oscillators: int = 4, coupling_K: float = 35.0, dt: float =
0.01):
self.num_oscillators = num_oscillators
self.K = coupling_K
self.dt = dt
self.phases = np.random.uniform(0, 2 * np.pi, size=num_oscillators)
self.natural_freqs = np.ones(num_oscillators) * (2 * np.pi * 1.0)

def sync_phases(self, steps: int = 10):
"""Pulls phases into full phase lock."""
for _ in range(steps):
d_phases = np.zeros_like(self.phases)
for i in range(self.num_oscillators):
interaction = np.sum(np.sin(self.phases - self.phases[i]))
```

```
d_phases[i] = (self.K / self.num_oscillators) * interaction
self.phases = (self.phases + d_phases * self.dt) % (2 * np.pi)

def update(self, perturbation: np.ndarray = None) -> float:
if perturbation is not None:
self.phases += perturbation

d_phases = np.zeros_like(self.phases)
for i in range(self.num_oscillators):
interaction = np.sum(np.sin(self.phases - self.phases[i]))
d_phases[i] = self.natural_freqs[i] + (self.K / self.num_oscillators) * interaction

self.phases = (self.phases + d_phases * self.dt) % (2 * np.pi)

# H4 CHORDLOCK snapping to 36° (pi/5) decagon grid
decagon_step = np.pi / 5.0
snapped_phases = np.round(self.phases / decagon_step) * decagon_step
diff = np.abs(self.phases - snapped_phases)
snap_mask = diff <= (decagon_step / 2.0)
self.phases[snap_mask] = snapped_phases[snap_mask]

pas = np.abs(np.mean(np.exp(1j * self.phases)))
return float(pas)

#
==========
=
# PRIMITIVE 3: AURA Legality Gate
#
==========
=
class AURALegalityGate:
```

```
def __init__(self, pas_threshold: float = 0.820):
self.threshold = pas_threshold

def evaluate(self, pas_score: float) -> dict:
is_legal = pas_score >= self.threshold
weight = 1.0 if is_legal else 0.0
channel = "Ring A (In-Phase)" if is_legal else "Ring B (Quadrature)"
isolation_db = 0.0 if is_legal else -149.9

return {
"pas_score": pas_score,
"is_legal": is_legal,
"weight": weight,
"active_channel": channel,
"isolation_db": isolation_db
}

#
==========================================================================
=
# PRIMITIVE 4: 19-Bit Universal Morphological Token (UMT) Codec
#
==========================================================================
=
class UMTCodec:
@staticmethod
def encode(chamber_id: int, phase_rad: float) -> int:
chamber_id = int(chamber_id) % 120
norm_phase = (float(np.mean(phase_rad)) % (2 * np.pi)) / (2 * np.pi)
phase_12bit = int(norm_phase * 4095) & 0xFFF
return (chamber_id << 12) | phase_12bit
```

```
@staticmethod
def decode(umt_token: int) -> tuple:
chamber_id = (umt_token >> 12) & 0x7F
phase_12bit = umt_token & 0xFFF
phase_rad = (phase_12bit / 4095.0) * (2 * np.pi)
return chamber_id, phase_rad

#
==========================================================================
=
# PRIMITIVE 5: 4D Quaternionic S^3 Hop Router
#
==========================================================================
=
class QuaternionicHopRouter:
def __init__(self):
self.vertices = self._generate_icosians()
self.adj_matrix = self._build_adjacency()

def _generate_icosians(self) -> np.ndarray:
verts = set()
phi = (1.0 + math.sqrt(5.0)) / 2.0

for i in range(4):
for s in [-1.0, 1.0]:
v = [0.0, 0.0, 0.0, 0.0]
v[i] = s
verts.add(tuple(v))
for w in [-0.5, 0.5]:
for x in [-0.5, 0.5]:
for y in [-0.5, 0.5]:
for z in [-0.5, 0.5]:
```

```
verts.add((w, x, y, z))

base = [phi / 2.0, 0.5, 1.0 / (2.0 * phi), 0.0]
even_perms = [
(0,1,2,3), (0,2,3,1), (0,3,1,2),
(1,2,0,3), (1,3,2,0), (1,0,3,2),
(2,0,1,3), (2,3,0,1), (2,1,3,0),
(3,0,2,1), (3,1,0,2), (3,2,1,0)
]
for p in even_perms:
for s0 in [-1.0, 1.0]:
for s1 in [-1.0, 1.0]:
for s2 in [-1.0, 1.0]:
v = [0.0, 0.0, 0.0, 0.0]
v[p[0]] = s0 * base[0]
v[p[1]] = s1 * base[1]
v[p[2]] = s2 * base[2]
v[p[3]] = 0.0
verts.add(tuple(v))

arr = np.array(list(verts))
norms = np.linalg.norm(arr, axis=1, keepdims=True)
return arr / norms

def _build_adjacency(self) -> dict:
adj = {}
target_dot = math.cos(math.pi / 5.0)
for i in range(len(self.vertices)):
dots = np.abs(np.dot(self.vertices, self.vertices[i]))
neighbors = np.where(np.abs(dots - target_dot) < 0.05)[0]
adj[i] = list(neighbors)
return adj
```

```
def route_hop(self, current_node_idx: int, target_q: np.ndarray) -> dict:
neighbors = self.adj_matrix.get(current_node_idx, [])
if not neighbors:
return {"selected_node": current_node_idx, "dot_product": 1.0,
"geodesic_distance_deg": 0.0}

best_node = current_node_idx
best_dot = abs(np.dot(self.vertices[current_node_idx], target_q))

for nbr in neighbors:
dot = abs(np.dot(self.vertices[nbr], target_q))
if dot > best_dot:
best_dot = dot
best_node = nbr

clamped_dot = min(1.0, max(0.0, best_dot))
geodesic_dist = math.degrees(math.acos(clamped_dot))

return {
"selected_node": best_node,
"dot_product": best_dot,
"geodesic_distance_deg": geodesic_dist
}

#
==========================================================================
=
# PRIMITIVE 6: Dual-Clock Thermodynamic Work Extractor
#
==========================================================================
=
```

```
class DualClockWorkExtractor:
def __init__(self, dt: float = 0.1):
self.dt = dt
self.binary_work = 0.0
self.virtual_work = 0.0
self.kuramoto = KuramotoPhaseIntegrator(num_oscillators=4)
self.gate = AURALegalityGate(pas_threshold=0.820)

def step(self, thermal_noise_std: float = 0.02) -> dict:
self.binary_work += 1.0 * (1.0 - thermal_noise_std)
pert = np.random.normal(0, thermal_noise_std, size=4) if thermal_noise_std > 0
else None
pas = self.kuramoto.update(perturbation=pert)
g_eval = self.gate.evaluate(pas)

if g_eval["is_legal"]:
hdof_gain = 2.115 * pas
self.virtual_work += 1.0 * hdof_gain
else:
self.virtual_work += 0.5 * pas

yield_ratio = self.virtual_work / max(0.001, self.binary_work)

return {
"binary_work": self.binary_work,
"virtual_work": self.virtual_work,
"yield_ratio": yield_ratio,
"pas": pas,
"is_legal": g_eval["is_legal"]
}

#
```

```
================================================================================
=
# INTERACTIVE REPL COMMAND SHELL
#
================================================================================
=
def print_header():
print("\033[96m\033[1m")
print("╔═══════════════════════════════════════════════════════════════════════
═══════════════════════════════════════╗")
print("║ SPIRALCORE CANTOR-ABRAXAS ARCHITECTURE CLI TERMINAL ║")
print("║ 6-Primitive Mutual Audit Engine & 4D Geodesic Router ║")
print("╚═══════════════════════════════════════════════════════════════════════
═══════════════════════════════════════╝")
print("\033[0m")
print("Type \033[1m'help'\033[0m for available commands, or
\033[1m'exit'\033[0m to quit.\n")

def print_help():
print("\033[1m=== AVAILABLE COMMANDS ===\033[0m")
print(" \033[96mstatus\033[0m - Display live health & audit metrics for all 6
primitives")
print(" \033[96msync [steps]\033[0m - Force Kuramoto limit-cycle phase lock
(locks PAS -> Ring A)")
print(" \033[96mstep [count] [noise]\033[0m - Execute N step transitions (e.g.
'step 10 0.02')")
print(" \033[96mroute [src] [dst]\033[0m - Trace 4D quaternionic geodesic route
on S³ (0..119)")
print(" \033[96mbenchmark [N]\033[0m - Benchmark 6-primitive pipeline
throughput & latency")
print(" \033[96mhelp\033[0m - Show this menu")
print(" \033[96mexit / quit\033[0m - Exit shell\n")
```

```
def run_repl():
print_header()

# Initialize Primitives
carry = ThreeAdicCarryEngine()
kuramoto = KuramotoPhaseIntegrator(num_oscillators=4)
gate = AURALegalityGate(pas_threshold=0.820)
router = QuaternionicHopRouter()
work_engine = DualClockWorkExtractor(dt=0.1)

current_node = 0

while True:
try:
cmd_input = input("\033[96m\033[1mspiralcore>\033[0m ").strip()
except (KeyboardInterrupt, EOFError):
print("\nExiting SpiralCore Terminal.")
break

if not cmd_input:
continue

parts = cmd_input.split()
verb = parts[0].lower()

if verb in ["exit", "quit"]:
print("Shutting down SpiralCore runtime.")
break

elif verb == "help":
print_help()
```

```
elif verb == "status":
pas = float(np.abs(np.mean(np.exp(1j * kuramoto.phases))))
g_eval = gate.evaluate(pas)
c_state = carry.step(0)
umt = UMTCodec.encode(c_state["beta_0"] * 15, kuramoto.phases)
w_state = work_engine.step(0)

print("\n\033[1m=== PRIMITIVES AUDIT STATUS ===\033[0m")
print(f"1. Carry Engine: \033[92mONLINE\033[0m | Step N={carry.N}, v3=
{c_state['v_3']}, β_0={c_state['beta_0']} (Bound ≤ {c_state['beta_0_bound']})")
print(f"2. Kuramoto Phase: \033[92mONLINE\033[0m | PAS={pas:.4f}")
print(f"3. AURA Gate: \033[92mONLINE\033[0m | Channel=
{g_eval['active_channel']} (Iso: {g_eval['isolation_db']} dB)")
print(f"4. UMT Codec: \033[92mONLINE\033[0m | Token:
\033[96m0x{umt:05X}\033[0m (2.375 Bytes)")
print(f"5. Quaternionic Hop: \033[92mONLINE\033[0m | Node=
{current_node}/120")
print(f"6. Dual-Clock Work: \033[92mONLINE\033[0m | Yield:
\033[95m{w_state['yield_ratio']:.3f}x\033[0m\n")

elif verb == "sync":
steps = int(parts[1]) if len(parts) > 1 else 10
kuramoto.sync_phases(steps=steps)
pas = float(np.abs(np.mean(np.exp(1j * kuramoto.phases))))
g_eval = gate.evaluate(pas)
print(f"\n\033[92m✓ Phase synchronization executed ({steps} steps).\033[0m")
print(f" New PAS: \033[92m{pas:.4f}\033[0m | Channel:
\033[92m{g_eval['active_channel']}\033[0m\n")

elif verb == "step":
count = int(parts[1]) if len(parts) > 1 else 5
```

```
noise = float(parts[2]) if len(parts) > 2 else 0.0

print(f"\n\033[1m=== EXECUTING {count} STEP TRANSITIONS (Noise={noise})
===\033[0m\n")
print(f"\033[1m{'Step':<6} | {'PAS':<8} | {'Gate Channel':<22} | {'β_0':<4} | {'4D
Hop':<12} | {'UMT Token':<10} | {'Work Yield'}\033[0m")
print("-" * 85)

target_q = router.vertices[85]

for s_i in range(1, count + 1):
c_state = carry.step(payload_value=float(s_i))
pert = np.random.normal(0, noise, size=4) if noise > 0 else None
pas = kuramoto.update(perturbation=pert)
g_eval = gate.evaluate(pas)

chamber_id = (c_state["beta_0"] * 15) % 120
umt = UMTCodec.encode(chamber_id, kuramoto.phases)

hop = router.route_hop(current_node, target_q)
current_node = hop["selected_node"]

w_state = work_engine.step(thermal_noise_std=noise)

channel_str = f"\033[92m{g_eval['active_channel']:<22}\033[0m" if
g_eval['is_legal'] else f"\033[91m{g_eval['active_channel']:<22}\033[0m"
pas_str = f"\033[92m{pas:.4f}\033[0m" if pas >= 0.820 else
f"\033[91m{pas:.4f}\033[0m"

print(f"{s_i:<6} | {pas_str:<17} | {channel_str} | {c_state['beta_0']:<4} | Node
{current_node:<7} | \033[96m0x{umt:05X}\033[0m |
\033[95m{w_state['yield_ratio']:.3f}x\033[0m")
```

```
print("\n\033[92m✓ Step execution completed.\033[0m\n")

elif verb == "route":
src = int(parts[1]) if len(parts) > 1 else 0
dst = int(parts[2]) if len(parts) > 2 else 85

print(f"\n\033[1m=== TRACING 4D GEODESIC ROUTE: Node {src} -> Node {dst}
===\033[0m\n")
target_q = router.vertices[dst]
curr = src

for hop_idx in range(1, 10):
hop = router.route_hop(curr, target_q)
print(f"Hop {hop_idx:<2} | Node {curr:<3} -> Node {hop['selected_node']:<3} |
Dot: {hop['dot_product']:.4f} | Distance: {hop['geodesic_distance_deg']:.2f}°")
if hop["selected_node"] == curr or hop["geodesic_distance_deg"] < 0.01:
print(f"\033[92m✓ Destination reached in {hop_idx} hops!\033[0m\n")
break
curr = hop["selected_node"]

elif verb == "benchmark":
n_iters = int(parts[1]) if len(parts) > 1 else 5000
print(f"\nBenchmarking {n_iters} transitions...")
t0 = time.time()
for _ in range(n_iters):
c = carry.step(1.0)
p = kuramoto.update()
g = gate.evaluate(p)
u = UMTCodec.encode(c["beta_0"], kuramoto.phases)
h = router.route_hop(0, router.vertices[10])
w = work_engine.step()
```

```
t1 = time.time()
dt = t1 - t0
ops_sec = n_iters / dt
print(f"\033[92m✓ Completed {n_iters} ops in {dt:.3f}s ({ops_sec:.2f}
ops/sec)\033[0m\n")

else:
print(f"Unknown command '{verb}'. Type 'help' for commands.\n")

if __name__ == "__main__":
run_repl()
```

Published with Simplenote

Report abuse